

Implementation Plan — Agentic Multimodal RAG

Status: Draft v0.9 (LangGraph agent, long-term memory phase; see spec §12) · **Date:** 2026-09-29 ·

Implements: [02-technical-specification.md](#) · **Governed by:** [01-constitution.md](#)

How this plan is executed

- Work proceeds **one phase at a time**. No code is written for a phase until the owner approves it (constitution W1, W2).
- Each phase ends with a **gate**: the owner reviews the deliverables against the acceptance criteria and signs off (W3).
- Phases 1–4 are built on **one PDF first** and extended to the full corpus in Phase 5. Phase 1 used Buildig-multimodal-rag.pdf (then all five PDFs were parsed and reviewed); from Phase 2 on, Transformers-in-Practice-Illustrated.pdf is the pilot, because its figures are drawn diagrams and charts.
- Effort estimates are rough, in focused working sessions, and will be refined after Phase 1.
- **Testing (constitution W5, LLD §10):** every phase's acceptance criteria also include:
 1. The **full automated test suite passes**.
 2. The phase's new behaviour is **covered by tests**, written *before* the code for deterministic logic, and written from the owner-approved results for exploratory parts.
 3. The phase summary reports the number of tests and the pass result.

Phase overview

Phase	Name	Main output	Needs GPU	Needs API key	Est. effort
0	Setup and decisions	venv, config, PostgreSQL + pgvector running, open decisions closed	–	✓ (verify)	1 session
1	Parsing	elements + figure PNGs + tables for 1 PDF	–	–	2–3 sessions
2	VLM captioning	captions for all figures of 1 PDF	✓ (Kaggle/Colab)	–	2 sessions
3	Enrichment and indexing	PostgreSQL schema + hybrid search working	–	✓ (if OpenAI embeddings)	1–2 sessions
4	Agent (LangGraph) and chart engine (CLI)	end-to-end multimodal answers in terminal/HTML; conversation threads	–	✓	3–4 sessions
5	Full corpus	all 5 PDFs ingested	✓	✓	1–2 sessions
6	Quality: rerank and evaluation	golden set, baseline scores, improvements	–	✓	2–3 sessions

Phase	Name	Main output	Needs GPU	Needs API key	Est. effort
7	UI and login	Streamlit chat app with email + password sign-in	–	✓	2 sessions
8	Hardening	incremental ingestion, logging, cost tracking, docs	–	✓	1–2 sessions
9	Long-term memory	per-user semantic and episodic memory, with user controls	–	✓	2 sessions

Phase 0: Setup and decisions

Goal: a working environment and all open decisions closed.

Tasks

- Owner confirms the open decisions D1–D6, D8 and D9 (spec §11). D1 (embedding model and dimension) must be settled here, because it fixes the database schema.
- Confirm an OpenAI **API key with billing** (separate from a ChatGPT subscription) and choose the agent model (D4).
- Create a **Python 3.11** virtual environment (3.11 is already installed; 3.12 would also work) and a dependency list.
- Create the folder structure (spec §8), `.env.example` and the config file.
- PostgreSQL + pgvector (spec §4, D9):**
 - Check the version of the existing Postgres install.
 - Run the official `pgvector/pgvector:pg16` Docker image through `docker-compose.yml`, on a different port if the existing Postgres uses 5432 (or add pgvector to the existing install).
 - Store the database files in a **named Docker volume**, not a bind-mounted Windows folder; Postgres fails to start on NTFS folder permissions.
 - Create the project database and user; put the credentials in `.env`.
- Create a Kaggle (or Colab) account and verify GPU access.
- Observability base (spec §7.7):**
 - Add the Phoenix trace viewer to `docker-compose.yml`.
 - Set up OpenTelemetry (traces, JSON logs with `trace_id`, metrics) in `mmrag/obs/`.
- Optionally initialize a git repository for version history.
- Retrofit tests (W5, added after Phase 0):** replace the manual checks with automated tests: config validation, JSON logs carrying trace IDs, telemetry failure safety, migrations (order and re-run), pgvector/HNSW, and `mmrag check` exit codes. A separate `mmrag_test` database is used; one `live` test covers OpenAI embeddings.

Deliverables: environment set up, config skeleton, decisions recorded in the spec.

Acceptance criteria

- The venv installs all dependencies without errors on Windows.
- A test API call to the chosen OpenAI model succeeds.

- A Kaggle or Colab notebook shows an available GPU.
- `CREATE EXTENSION vector;` succeeds in the project database, and a test HNSW index can be created.
- The database container survives a restart with its data intact (named volume).
- A test trace, with a correlated log line, appears in Phoenix.

Gate: owner approves before Phase 1.

Phase 1: Parsing (one PDF)

Goal: capture everything from Buildig-multimodal-rag.pdf with provenance and no silent loss.

Tasks

1. **Corpus profiler:** a report per PDF covering pages, images, vector-heavy pages, tables and textless pages, for all 5 PDFs.
2. **Text extraction:** blocks with bounding boxes, heading detection and `section_path`, and header/footer removal with a log.
3. **Embedded image extraction:** native resolution, de-duplication, decorative filter with logged reasons.
4. **Table extraction (before figure detection):** structured tables, plus an image fallback for low-confidence tables.
5. **Vector figure detection** (spec §6.1):
 - Remove table regions, header and footer rules, page borders and isolated simple shapes.
 - Treat text-only boxes as callouts, not figures.
 - Keep box-and-arrow diagrams.
 - Cluster, render at 200 DPI or higher, and attach "Figure N" captions.
 - All thresholds come from config.
6. **Textless page detection:** render whole-page PNGs for VLM transcription.
7. **Review sheet:** a simple HTML page showing every detected figure and table next to its page, so the owner can check detection quality by eye. It also lists regions that were **rejected** by the filters, so wrongly discarded diagrams are visible.
8. **Threshold tuning:** adjust the clustering and filter thresholds using the review sheet. Target cases: table grids mistaken for diagrams, two diagrams merged into one, and box-and-arrow diagrams wrongly dropped. Record the final values in config.
9. **Tracing:** each parse run is one `ingest.document` trace with a span per step. Page, element and skip counts are span attributes.
10. **Tests (W5):**
 - **Fixture pages first:** sample pages cut from the PDFs (a table grid, a box-and-arrow diagram, a textless page, and a page from each other PDF), each with expected element counts written **before** the parser.
 - Unit tests for IDs, heading detection, the furniture and isolated-shape filters.
 - After threshold tuning, the owner-approved review-sheet results are frozen as regression tests.

Deliverables: `elements.jsonl`, `assets/*.png`, `tables.jsonl`, profile report, skip log, review sheet.

Acceptance criteria

- Every page has at least one element, or a logged reason why not.
- Owner review: at least 90% of real figures detected as whole figures (not split or merged); no obvious figure missing.
- No table grid or header rule is detected as a figure.
- Tables in the sample appear as correct rows and columns.
- Every element has file, page, bbox and ID.

Gate: owner reviews the review sheet and approves.

Phase 2: VLM captioning (one PDF)

Goal: high-quality structured captions for every figure, produced on a temporary GPU.

Tasks

1. **Bundle exporter:** zip the figure PNGs and `jobs.jsonl` with page context.
2. **Captioning prompt v1:** type-aware instructions and a JSON schema (spec §5.2). Test first on 5 sample figures.
3. **Kaggle/Colab notebook with three model paths** (spec §6.2):
 - **Primary:** Qwen2.5-VL-7B AWQ on vLLM in fp16, with guided JSON.
 - **Fallback A:** 7B in 4-bit NF4 via `transformers` + `bitsandbytes`, using JSON mode with pydantic validation and JSON repair.
 - **Fallback B:** 3B in fp16.

The notebook applies the `max_pixels` image cap, resumes after interruption and writes `captions.jsonl`.

4. **Pilot run on 10 figures with each path that installs and fits in GPU memory.** Record GPU memory use, speed and JSON validity. The owner compares caption quality side by side and **chooses the model path**.
5. **Prompt and `max_pixels` iteration** if needed. Raise the cap only for dense diagrams with unreadable labels.
6. **Full run** for all figures of the PDF, on the chosen path.
7. **Cache** results by content hash, model ID and prompt version.

Deliverables: GPU notebook (all three paths), prompt file (versioned), pilot comparison report, `captions.jsonl`.

Acceptance criteria

- 100% of non-decorative figures have a schema-valid caption, or are flagged `needs_review`.
- Owner spot-check of 10 captions: at least 8 judged accurate and complete, meaning all key labels and the correct flow.
- Chart figures carry extracted values with exact/estimated flags.
- Re-running the job on the same bundle makes zero model calls, thanks to the cache.

Gate: owner approves caption quality.

Phase 3: Enrichment and indexing

Goal: make text, figures and tables searchable.

Tasks

1. **Caption import:** validate captions and mark failures `needs_review`.
2. **Label cross-check** against PDF text inside vector-figure regions.
3. **Figure and table linking** (spec §6.3), in priority order:
 1. Explicit "Figure N" / "Table N" references.
 2. Phrases such as "as illustrated below".
 3. Spatial-proximity fallback: the preceding text block on the same page, or within about ± 150 points.
4. **Table summaries:** `summarize_tables()` (LLD §3.5) writes a 2-sentence synthetic summary per table using `agent.summary_model`, cached by table hash.
5. **Section-aware chunking.** Table documents are split three ways: summary + headers for semantic search (`dense_text`), raw cells for keyword search (`keyword_text`), structured JSON for `get_table`.
6. **Embeddings** (per D1), cached by hash.
7. **Database schema and write path** (spec §5.5, §6.5):
 - Versioned SQL migrations for all tables.
 - HNSW indexes on `embedding` (partial per collection); generated `tsv_english` and `tsv_simple` columns with GIN indexes.
 - **One transaction per document** (insert new version → insert all rows → delete old version → commit).
8. **Hybrid search query** (spec §7.1): semantic top 30 + keyword top 30 → RRF → top k , in one SQL query per collection. Element locations are aggregated into one array per chunk, so each chunk is one row.
9. **Retrieval test script:** a fixed list of about 15 queries, showing the top results per collection. It includes at least 3 table-targeted queries phrased in words, not cell values, and at least 3 exact-term queries (acronyms such as "GSM8K" or "LoRA") to check the `simple` keyword index.
10. **Tracing:** enrich, chunk, embed and the database write are spans in the same ingestion trace. LLM and embedding calls carry model, tokens and cost.
11. **Tests (W5), test-first:**
 - chunk boundaries (never crossing a section), and chunk and element IDs;
 - figure-linking rules, and table document splitting;
 - RRF scoring, and "each chunk appears once";
 - transactional writes: a failure injected mid-document leaves the old version intact; an unchanged re-ingest is a no-op; cascade delete.

These run against the `mmrag_test` database.

Deliverables: migrations, populated PostgreSQL database, hybrid search query, retrieval test report.

Acceptance criteria

- For about 15 hand-written test queries, the expected chunk or figure appears in the top 5 at least 80% of the time.

- Every search hit resolves to a valid citation and asset.
- Every figure has at least one linked text block, or is listed as unlinked in a report.
- **Consistency test:** an ingestion deliberately interrupted mid-document leaves the previous version fully searchable and no partial rows.
- Re-ingesting an unchanged PDF writes nothing.
- Every search returns each chunk **exactly once**, with its element locations in reading order.

Gate: owner reviews the retrieval test report.

Phase 4: Agent (LangGraph) and chart engine (CLI first)

Goal: end-to-end multimodal answers, before building any UI.

Tasks

1. **Tool implementations:** `search_text`, `search_figures`, `search_tables`, `get_figure`, `get_table`, `view_page`, `compute`.
2. **Chart engine:** `make_chart` with validation rules (spec §7.4); Plotly figures plus PNG export.
3. **Answer schema:** block format (spec §5.4) through structured output. The agent cites by `id` only: `element_id` for figures and tables, `chunk_id` for text.
4. **Agent graph (LangGraph, D3):** a `StateGraph` with nodes `plan` → `agent` ⇌ `tools` → `compose` → `validate` → (repair). Nodes call the OpenAI SDK directly; tools are plain functions; the LangGraph version is pinned.
 - A **Postgres checkpointer** keeps each conversation as a thread (`thread_id`), so follow-up questions use earlier turns (FR-23). `mmrag ask` takes an optional `--thread`.
 - **Agent model comparison:** the 10 sample questions with `gpt-4o-mini`, `gpt-5.4-mini` and one larger model, side by side with cost, latency and validator results; the owner picks (D4).
 - Loop behaviour:
 - **Parallel tool calls** are enabled.
 - The planner tags each question; multi-part or comparison questions get up to 5 rounds, all others 3.
 - When the limit is reached, the agent answers with what it has and states what is missing.
5. **Answer validator:** constitution checks with one repair attempt. It also **hydrates each citation** with `source_file`, `page` and `bbox` from PostgreSQL, expands chunk citations into their elements, and rejects unknown IDs.
6. **Output renderer:** a CLI command that writes each answer to a local HTML file, with text, images, charts and tables, for viewing in the browser.
7. **Tracing and logging (spec §7.7, LLD §5.9):**
 - One trace per question: plan, each round, each tool call, LLM calls, SQL searches, chart rendering, validation, repair.
 - `trace_id`, rounds used, latency, tokens and cost stored in `query_log`.
8. **Tests (W5), test-first:**
 - `compute` whitelist, and chart validation (values ∈ evidence, pie → bar fallback, "approximate" flag);
 - validator rules and citation hydration, and `bbox` conversion;
 - tool argument validation;

- graph termination, round limits and the repair path, using a **mocked OpenAI client** (no API cost);
- thread continuity: a follow-up in the same thread sees the previous turn; a new thread does not.

Deliverables: working agent from the CLI; HTML answer output; per-query cost log.

Acceptance criteria

- 10 sample questions (text, visual and quantitative) produce correct block types:
 - Visual questions include the original figure.
 - Quantitative questions include a chart plus its data table.
- **No chart contains a value missing from its cited source** (manual check of all charts).
- An unanswerable question gets an honest "not found in the documents" answer.
- Median cost per question is at most US\$0.15, and median latency is at most 20 s (provisional, pending Phase 6).
- All citations carry validator-hydrated `bbox` values; none come from the LLM.
- Every question produces exactly one trace with the full step tree, and its `trace_id` in `query_log` opens it in Phoenix.

Gate: owner reviews 10 answers.

Phase 5: Full corpus

Goal: all 5 PDFs ingested.

Tasks

1. Run Phase 1 parsing on the remaining 4 PDFs and review their review sheets. Pay special attention to the textless pages in the LLM Lifecycle Notes.
2. Run one GPU captioning session for all remaining figures and textless pages.
3. Import captions and write each new document to PostgreSQL (one transaction each); existing documents are untouched.
4. Re-run the Phase 3 retrieval tests plus about 10 new cross-document queries.

Acceptance criteria: same as Phases 1–3, applied to every PDF; cross-document questions retrieve from the correct files.

Gate: owner approval.

Phase 6: Quality — reranking and evaluation

Goal: measured quality and a baseline for future changes.

Tasks

1. **Golden set (spec §9):**
 - Optionally draft candidate questions with RAGAS's test-set generator (`eval draft-questions`).

- Write the final 40 questions with the owner. Each has a reference answer, reference evidence IDs, expected figures, chart values and tool calls, and an answerable flag.

2. Evaluation runner (LLD §5.10):

- RAGAS (pinned version) with the judge model (D14): faithfulness, multimodal faithfulness, response relevancy, context precision and recall, factual correctness, tool call accuracy.
- Custom metrics: chart numeric correctness, figure hit rate, citation accuracy, refusal, latency and cost.
- Results stored in `eval_runs` / `eval_results`; a report with trace links; a regression gate against the baseline.

3. Run the **baseline** evaluation twice, to measure judge variance. The owner spot-checks 10 judgments.

4. Add the local cross-encoder reranker and query rewriting; re-evaluate and keep only changes that improve the scores (P10).

5. Tune the prompts (agent, caption) against the evaluation.

6. **Keyword ranking check (D8):** compare built-in full-text ranking with ParadeDB `pg_search` (BM25) on the golden set. Adopt `pg_search` only if it measurably improves retrieval. Also tune HNSW `ef_search` and the RRF candidate counts.

7. **Finalize the round limits and latency/cost targets together:** compare 3 vs 4 vs 5 rounds (default and multi-part) on quality, latency and cost. Record the final values in spec §7.2 and NFR-1/NFR-2.

Deliverables: golden set, evaluation runner, baseline and improved score reports.

Acceptance criteria: meets the v1 targets in spec §9, or the owner accepts documented gaps.

Gate: owner approval.

Phase 7: UI and login

Goal: a usable chat application, available only to signed-in users.

Tasks

1. Streamlit chat layout with conversation history. Each chat is a LangGraph thread (from Phase 4), so a user can reopen and continue an earlier conversation. Store each finished answer (hydrated blocks and chart figures) in `st.session_state`. Streamlit re-runs the script on every click, so viewing a source or expanding a table must render from stored state, never re-run the agent, validator or database queries. Rendered page images are cached.
2. Block renderers:
 - Markdown text with citation chips.
 - Original images with captions and click-to-enlarge.
 - Interactive Plotly charts.
 - Data tables, collapsible under each chart.
3. Source panel: click a citation to see the page image with the element highlighted.
 - The page is rendered at a known DPI.
 - `bbox` is converted from PDF points to pixels ($\times \text{DPI}/72$), with rotated pages handled (spec §7.5).
 - The highlight is drawn directly from the hydrated citation, with no extra lookup.
4. Progress indicator showing the agent's steps ("searching figures..."), streamed from the LangGraph graph.

5. Sidebar with per-session cost and token counter.

6. **Sign-in (spec §7.6, LLD §5.8):**

- Migrations for `users`, `sessions`, `auth_events`, plus `user_id` and `answer_json` on `query_log`.
- Sign-in and change-password screens; `require_user()` on every rerun; logout.

7. **Account admin CLI:** `user add`, `reset-password`, `disable`, `enable`, `unlock`, `list`.

8. **Protection:**

- Argon2id hashing, and a password policy with a common-password list.
- Account lockout and per-IP throttling, with the same error message for every failure.
- Audit events.

9. **Per-user limits and history:** daily question and cost limits are checked before each question; each user sees only their own chat history.

10. **Admin metrics page (FR-22):** questions per day, error rate, slowest answers (linked to their traces), cost per user, failed sign-ins.

11. **Tests (W5), test-first:**

- password hashing and policy;
- same message for a wrong password and an unknown email;
- lockout and release, idle and absolute session expiry;
- revocation on logout and password change;
- daily limits, and no secrets in logs.

Plus a UI smoke test.

Acceptance criteria

- All Phase 4 sample answers render correctly in the UI, and sources open the correct page.
- Nothing but the sign-in screen is shown until a user signs in.
- A wrong password and an unknown email produce the same message; 5 failures lock the account for 15 minutes.
- Logout and password change end the session; a disabled user cannot sign in.
- A user at the daily limit gets a clear message, and no API call is made.
- No password or session token appears in plain text in the logs or the database.

Gate: owner demo and approval.

Phase 8: Hardening

Tasks

1. Ingestion commands: **add** a PDF, **replace** a PDF with a new version (transactional swap), and **remove** a PDF (cascade delete). Each processes only that PDF.
2. Error handling and retries for API calls; clear error messages.
3. Configuration documentation and README (setup, ingestion, GPU job, running the app).
4. **Database operations:** backup and restore procedure (`pg_dump` / managed snapshots), with a tested restore; documented blue-green procedure for changing the embedding model (spec §6.5); notes for

moving to managed Postgres with pgvector in production.

5. Security hardening for deployment:

- HTTPS through a reverse proxy (TLS), with Streamlit listening only on localhost.
- Proxy headers trusted only behind that proxy.
- Review of the sign-in audit events, and a documented procedure for admin password recovery.

6. Production observability:

- OpenTelemetry Collector with batching, and sampling that keeps every error and slow trace; content capture off.
- Retention cleanup command (`obs cleanup`).
- Alerts for daily cost over budget, an error-rate spike and many failed sign-ins.

7. **Continuous integration:** a GitHub Actions workflow runs the unit and integration tests (Postgres + pgvector service) on every push and pull request; `live` tests stay manual.

8. Final evaluation run and a short results report.

Acceptance criteria: a fresh setup following the README works end to end, and adding a 6th PDF works without re-processing the existing five, and a database restore from backup returns identical search results. When deployed, the app is reachable only over HTTPS, and stopping the trace viewer does not affect answers.

Phase 9: Long-term memory

Goal: the assistant remembers each user across conversations (spec §7.8, constitution P13), without memory ever becoming evidence.

Tasks

1. **Store:** LangGraph Store on the project PostgreSQL with pgvector search; namespaces per user and memory kind (D15).
2. **Semantic memory:** after a validated answer, a low-cost model extracts stable facts and preferences as short statements; an existing statement on the same subject is updated, not duplicated.
3. **Episodic memory:** when a thread ends (or goes idle), a summary of what was asked, found and left open is stored.
4. **Recall:** a `recall_memory` node retrieves the user's most relevant memories before planning and passes them as user context.
5. **Validator rule:** no citation to a memory; no answer fact supported only by memory (P13).
6. **User controls (FR-25):** list and delete memories, switch memory off (CLI first, then a UI page); account deletion removes memories.
7. **Privacy and retention (NFR-13):** memory text out of production telemetry; retention in config; cleanup command.
8. **Evaluation:** RAGAS faithfulness unchanged with memory on; a small follow-up set checks that memory resolves references ("the chart you showed me last week") and preferences.
9. **Tests (W5), test-first:** extraction and update rules, namespace isolation between users, deletion, the P13 validator rule, recall on a mocked store.

Acceptance criteria

- A preference stated in one conversation is applied in a later one.
- A follow-up that refers to an earlier conversation is understood.
- One user can never see another user's memories.
- Deleted memories are no longer recalled; memory switched off means none are stored or recalled.
- Faithfulness and citation accuracy do not drop with memory on (Phase 6 baseline).

Gate: owner demo and approval.

Milestones

Milestone	Reached after	What the owner can do
M1: Inspectable corpus	Phase 1	Browse every extracted figure and table for one PDF
M2: Understood figures	Phase 2	Read accurate captions for every figure
M3: First multimodal answer	Phase 4	Ask a question and get text + original image + chart (in HTML)
M4: Whole library	Phase 5	Ask across all 5 PDFs
M5: Measured quality	Phase 6	See scores and trust the chart numbers
M6: Usable app	Phase 7	Sign in and chat in a browser UI
M7: Assistant that remembers	Phase 9	Continue across conversations; see and delete what it remembers

Cost expectations (estimates)

Item	Estimate
GPU captioning (Kaggle/Colab)	Free, or about US\$1–5 if rented
Embeddings, full corpus (OpenAI)	Under US\$1 one-time
PostgreSQL + pgvector (local Docker)	Free; managed Postgres in production depends on the provider and size
OpenTelemetry + Phoenix / Jaeger (self-hosted)	Free
Development and testing queries	About US\$5–20 total, depending on the model
Evaluation runs	About US\$3–10 per full 40-question run (agent answers + RAGAS judge calls); measured on the first run
Ongoing use	About US\$0.05–0.15 per question

Next step

Phases 0–3 are complete. Next: the owner approves the **Phase 4** plan (LangGraph agent, chart engine, validator, answer rendering, agent model comparison).